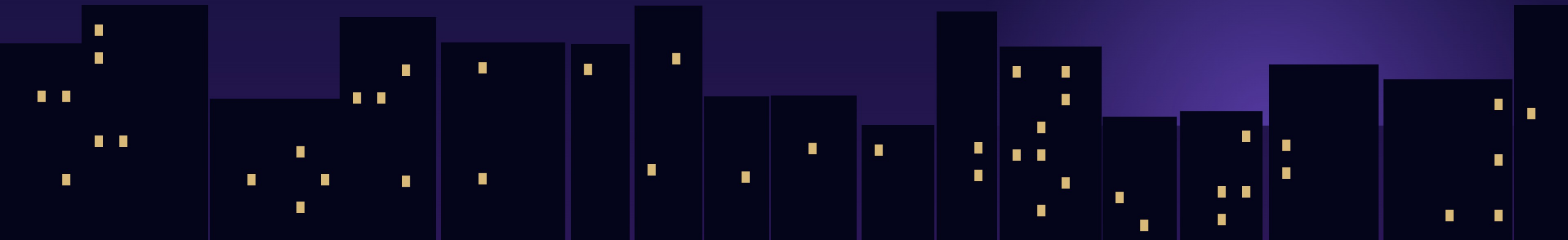
DESENVOLVIMENTO FRONTEND · MARCO 2

Os Rapazes da Noite

Lista de Tarefas em Angular

Artur Wanderley · André Carvalho · Ricardo Barbosa · Paulo Martins



O QUE CONSTRUÍMOS

Cinco telas, cada uma com seu endereço

`/tarefas`	`/tarefas/:id`	`/projetos`	`/projetos/:id`	`**`
Lista	**Detalhe**	**Projetos**	**Projeto**	**404**
Todas as tarefas da API, com busca e filtros	Todos os campos de uma tarefa	Lista de projetos, cada um com link	Dados do projeto e as tarefas dele	Qualquer endereço que não existe

Tudo lendo dados reais da API local (json-server) e trocando de tela **sem recarregar a página**.
Nos próximos slides: **como** cada parte foi feita.

COMO FIZEMOS · 1 DE 4

Os dados chegam da API

Quem fez: André

```
// services/api.ts
async listarTarefas(): Promise<Tarefa[]> {
const resposta = await fetch(`${API}/tarefas`);
if (!resposta.ok) throw new Error(...);
return resposta.json();
}
```

```
// pages/lista-tarefas.ts
tarefas = signal<Tarefa[]>([]);
carregando = signal(true);
erro = signal<string | null>(null);
```

Um serviço só para a API

Toda chamada ao json-server passa pelo Api, tipado com o contrato Tarefa.

A página guarda o estado

Três signals: as tarefas, se está carregando e se deu erro.

A tela segue o estado

O @if mostra carregando, erro ou a lista. O Angular só atualiza o que mudou.

COMO FIZEMOS · 2 DE 4

A busca filtra sozinha

Quem fez: Artur

```
// lista-tarefas.ts (resumido)
busca = signal('');
statusSelecionado = signal('');

tarefasFiltradas = computed(() ⇒
this.tarefas().filter((tarefa) ⇒
tarefa.titulo.toLowerCase()
.includes(this.busca().toLowerCase())
&& (!this.statusSelecionado()
|| tarefa.status ⩵ this.statusSelecionado())
)
);
```

Cada campo é um signal

Texto, status e prioridade. Digitar só atualiza o signal.

computed recalcula

Quando um filtro muda, a lista filtrada é refeita sem ninguém chamar nada.

A original fica intacta

O filter cria uma lista nova. Limpar a busca traz tudo de volta.

COMO FIZEMOS · 3 DE 4

O roteador liga as telas

Quem fez: Paulo · Artur

```
// app.routes.ts
{ path: '', redirectTo: 'tarefas' },
{ path: 'tarefas', component: ListaTarefas },
{ path: 'tarefas/:id', component: DetalheTarefa },
{ path: 'projetos', component: Projetos },
{ path: 'projetos/:id', component: DetalheProjeto },
{ path: '**', component: NaoEncontrada },

// detalhe-tarefa.ts
const id = this.rota.snapshot.paramMap.get('id');
```

Sem recarregar

O routerLink troca a tela pela History API. O Voltar do navegador funciona.

O :id vem da URL

O detalhe lê o id e busca só aquela tarefa ou projeto na API.

O ** por último

O coringa pega o que sobrou e mostra a 404. Antes, engoliria as outras rotas.

COMO FIZEMOS · 4 DE 4

Um cartão em todo lugar

Quem fez: André · Ricardo

```
// tarefa-card.ts: o filho declara a entrada
tarefa = input.required<Tarefa>();
```

```
// lista e página do projeto: o pai entrega
@for (tarefa of p.tarefas; track tarefa.id) {
<app-tarefa-card [tarefa]="tarefa" />
}
```

```
// tarefa-card.spec.ts: no teste, o teste é o pai
fixture.componentRef.setInput('tarefa', tarefaExemplo);
```

O dado desce

A página é dona das tarefas. O cartão recebe uma e não altera a prop.

Escrito uma vez

"Em andamento" e "06/10/2026" saem iguais na lista e no projeto.

Testado

Todos os testes passam: 7 de 7.

FIM DO MARCO 2

Obrigado e boa noite!

Os Rapazes da Noite

